

MANUAL TÉCNICO

Sistema de Control de Garita (FastAPI)

1. Introducción

Este manual técnico describe la arquitectura, funcionamiento e implementación del Sistema de Control de Garita desarrollado con FastAPI y PostgreSQL. Está dirigido a desarrolladores o personal técnico encargado del mantenimiento del sistema. El sistema de control de garita fue desarrollado utilizando arquitectura cliente-servidor. El backend fue implementado utilizando FastAPI y Python, mientras que PostgreSQL fue utilizado como sistema gestor de base de datos relacional.

El frontend fue desarrollado utilizando HTML, CSS y JavaScript para crear una interfaz dinámica e intuitiva.

SQLAlchemy fue utilizado como ORM para facilitar la conexión entre Python y PostgreSQL.

2. Descripción del sistema

El sistema es una API REST desarrollada en Python utilizando FastAPI. Permite la gestión de visitantes y vehículos mediante operaciones CRUD (Create, Read, Update, Delete).

3. Arquitectura del sistema

El sistema implementa una arquitectura dividida en:

Frontend

Interfaz gráfica desarrollada en:

- HTML5
- CSS3
- JavaScript

Backend

Implementado utilizando:

- FastAPI
- Python

Base de Datos

Utiliza PostgreSQL para almacenar información de:

- Usuarios.
- Vecinos.
- Visitantes.
- Historial.
- Vehículos.

4. Tecnologías utilizadas

Lenguaje de Programación

- Python 3.11

Framework Backend

- FastAPI

ORM

- SQLAlchemy

Servidor ASGI

- Uvicorn

Base de Datos

- PostgreSQL

Librerías

- qrcode
- pillow
- bcrypt
- python-jose
- jinja2

5. Estructura del proyecto

El proyecto está organizado en módulos:

- main.py → Punto de entrada
- models/ → Modelos de base de datos
- routes/ → Endpoints de la API

- database/ → Configuración de conexión
- schemas/ → Validación de datos

6. Base de datos

Se utiliza PostgreSQL como gestor de base de datos.

Tablas principales:

- visitantes
- vehiculos
- registros

SQLAlchemy se utiliza como ORM para facilitar la interacción con la base de datos.

7. Endpoints principales

El sistema incluye endpoints REST:

- POST /visitantes → Registrar visitante
- GET /visitantes → Listar visitantes
- POST /vehiculos → Registrar vehículo
- GET /vehiculos → Consultar vehículos

8. Implementación frontend

La interfaz fue desarrollada utilizando plantillas HTML.

Archivos principales

- login.html
- dashboard.html
- visitantes.html
- vecinos.html

9. Instalación del sistema

Para instalar el sistema:

```
git clone <repositorio>
cd proyecto
python -m venv venv
```

```
venv\Scripts\activate  
pip install -r requirements.txt
```

10. Ejecución del sistema

Para ejecutar el servidor:

```
uvicorn main:app --reload
```

Acceder a:

```
http://127.0.0.1:8000/docs
```

11. Validación de datos

Se utiliza Pydantic para validar los datos de entrada, asegurando que los campos obligatorios sean correctos antes de almacenarlos en la base de datos.

12. Seguridad del sistema

- Validación de datos
- Separación de capas (modelo, rutas, esquema)
- Uso de variables de entorno para credenciales
- Posible implementación futura de JWT

13. Mantenimiento

Se recomienda:

- Respalidar la base de datos regularmente
- Mantener dependencias actualizadas
- Revisar logs del sistema
- Optimizar consultas SQL si es necesario

14. Escalabilidad

El sistema puede ampliarse para incluir:

- Autenticación de usuarios
- Integración con cámaras
- Sistema de QR para visitantes
- Panel web administrativo

15. Problemas comunes

- Error de conexión a base de datos
- Puertos ocupados en FastAPI
- Dependencias faltantes

16. Pruebas del sistema

Se realizaron pruebas de:

- Registro usuarios.
- Registro visitantes.
- QR.
- Correos.
- Base de datos.
- Consultas SQL.

17. Conclusión técnica

El sistema está diseñado bajo buenas prácticas de desarrollo de software, permitiendo escalabilidad, mantenimiento y futuras mejoras sin afectar la estructura principal, demuestra la integración efectiva entre FastAPI, PostgreSQL y tecnologías frontend modernas.

La arquitectura cliente-servidor permite escalabilidad, seguridad y facilidad de mantenimiento. Además, el uso de SQLAlchemy facilita la administración de la base de datos y mejora el desarrollo del sistema.

La integración de QR y correos electrónicos proporciona funcionalidades modernas que fortalecen el control de acceso residencial.

AUTORA

Shelsi Coronado

Sistema: Control de Garita

Tecnologías: FastAPI + PostgreSQL + Python